



Program Studi Bioinformatika
FMIPA IPB University

Modul Praktikum

Sistem Basis Data Bioinformatika

Pertemuan 4

SQL Dasar

Prof. Dr. Eng. Wisnu Ananta Kusuma, S.T., M.T.
Dr. Luthfan Irfana, S.Si., M.Si.
Lailan Sahrina, Hasibuan, S.Kom., M.Kom.
Gilland Fausta Putra Achyar, S.Mat., M.Kom.
Said Thaufik Rizaldi, S.Kom., M.Kom.
Hakim Ilyas Azhar
Tsabitha Naylasafa Aurora

Daftar Isi

1	Pendahuluan	1
2	Pengenalan Data Biologis	1
	2.1 Konteks Dataset	1
	2.2 Informasi Gen	1
	2.3 Anotasi Pathway	2
	2.4 Mengenali Tiga Berkas Data Praktikum	3
3	Model Data Praktikum.	3
	3.1 ERD Praktikum	3
	3.2 Kamus Data Praktikum	4
4	Tugas Prapraktikum	4
5	Konsep SQL Dasar	5
	5.1 Perintah SQL yang Digunakan	5
	5.2 Struktur dan Constraint	5
6	Persiapan Lingkungan Praktikum.	5
	6.1 Perangkat dan Ruang Kerja	5
	6.2 Berkas Praktikum Mahasiswa	6
	6.3 Membuka Query Tool dan Menyiapkan Database	6
7	Penerapan SQL Dasar	6
	7.1 Membuat Tabel dengan CREATE TABLE	6
	7.2 Menambahkan Data Awal dengan INSERT	8
	7.3 Membaca Data dengan SELECT	9
	7.4 Mengubah Struktur dan Data dengan ALTER TABLE dan UPDATE	10
	7.5 Menambahkan Data Simulasi dan Hubungannya	11
	7.6 Menghapus Data dengan DELETE	12
	7.7 Menguji Integrity Constraint	13
8	Latihan Individual dan Refleksi	14
	8.1 Tantangan Individual	14
	8.2 Refleksi Prediksi dan Hasil	15
9	Lembar Kerja Praktikum	15
	9.1 Berkas yang Dikumpulkan	15
	9.2 Isi Skrip SQL	15
	9.3 Struktur Laporan Praktikum	15
	9.4 Ketentuan Teknis	16
10	Daftar Pustaka	17

Tujuan Praktikum

1. Membuat dan mengubah struktur tabel data biologis menggunakan `CREATE TABLE` dan `ALTER TABLE`.
2. Menambahkan, menampilkan, memperbarui, dan menghapus data menggunakan `INSERT`, `SELECT`, `UPDATE`, dan `DELETE`.
3. Menjalankan skrip SQL serta memeriksa hasil operasi dan penerapan *constraint* pada PostgreSQL.

1 Pendahuluan

Pada pertemuan sebelumnya, rancangan basis data telah dipetakan menjadi tabel, atribut, *primary key*, *foreign key*, dan *integrity constraint*. Rancangan tersebut selanjutnya diwujudkan dan dikelola pada *database management system* (DBMS) menggunakan *Structured Query Language* (SQL).

Praktikum ini menggunakan data biologis berupa katalog gen *Plasmodium falciparum* 3D7 dan anotasi *pathway*. Data tersebut menyimpan identitas gen, lokasi gen pada sekuens referensi, arah *strand*, serta hubungan gen dengan *pathway*. Dataset yang digunakan merupakan subset gen yang berkaitan dengan tiga *pathway*, sehingga hasil pengamatan dan interpretasi hanya berlaku dalam subset ini.

Mahasiswa akan menggunakan `CREATE TABLE` untuk membuat tabel, `ALTER TABLE` untuk mengubah struktur tabel, serta `INSERT`, `SELECT`, `UPDATE`, dan `DELETE` untuk mengelola data. Setiap perintah dijalankan secara bertahap agar perubahan pada struktur dan isi basis data dapat diamati.

2 Pengenalan Data Biologis

2.1 Konteks Dataset

Praktikum menggunakan subset katalog gen *Plasmodium falciparum* 3D7 yang dipilih berdasarkan tiga *pathway*. Data memuat lokasi gen pada sekuens referensi dan anotasi gen-*pathway*. Karena bukan sampel acak seluruh genom, hasil *query* hanya ditafsirkan dalam cakupan subset ini.

2.2 Informasi Gen

Data gen memuat `gene_id`, `seqid`, `start_pos`, `end_pos`, dan `strand`. `gene_id` membedakan setiap gen; gabungan `seqid`, `start_pos`, dan `end_pos` menunjukkan lokasinya pada sekuens referensi; sedangkan `strand` menyatakan orientasi gen.

Sekuens Referensi dan Pengenal Gen

Kolom `gene_id` menyimpan pengenal gen, misalnya PF3D7_0815900, sedangkan `seqid` menyimpan label sekuens referensi, misalnya Pf3D7_08_v3. Keduanya mewakili informasi berbeda. Label `seqid` disimpan utuh sebagai teks karena koordinat gen baru bermakna jika sekuens referensinya diketahui. Dataset hanya mencakup satu organisme dan satu *snapshot*, sehingga model ini belum memerlukan tabel organisme atau riwayat versi referensi.

Gen, CDS, dan Protein

Gen adalah bagian DNA yang informasinya digunakan untuk menghasilkan produk fungsional berupa RNA atau, melalui RNA, protein. Pada gen pengode protein, DNA ditranskripsi menjadi RNA; intron dikeluarkan melalui *splicing*, sedangkan ekson dipertahankan dalam mRNA matang. *Coding DNA sequence* (CDS) adalah bagian yang diterjemahkan menjadi protein. Rentang gen juga dapat mencakup intron dan bagian lain yang tidak diterjemahkan, sehingga panjangnya tidak otomatis sama dengan panjang CDS, mRNA matang, atau protein.

Lokasi gen dibaca menggunakan `seqid`, `start_pos`, dan `end_pos`. Koordinat menggunakan sistem *1-based inclusive*, sehingga posisi awal dan akhir sama-sama dihitung. Panjang rentang genom adalah:

$$\text{panjang rentang} = \text{end_pos} - \text{start_pos} + 1.$$

Nilai + dan - pada atribut `strand` menunjukkan arah gen terhadap sekuens referensi. Strand negatif tidak berarti gen rusak, tidak aktif, atau kurang penting.

2.3 Anotasi Pathway

Reaksi Metabolisme dan Peran Protein

Metabolisme mencakup reaksi kimia yang mengubah molekul di dalam sel. Banyak reaksi dikatalisis oleh enzim, yang umumnya merupakan protein hasil ekspresi gen. *Pathway* merepresentasikan reaksi atau proses biologis yang saling berkaitan dan membantu mengorganisasi informasi mengenai fungsi biologis.

Tabel 1. Tiga *pathway* dalam dataset praktikum.

Identifier	Nama ringkas	Pengertian dasar
pfa00010	Glikolisis/Glukoneogenesis	Glikolisis berkaitan dengan pemecahan glukosa; glukoneogenesis berkaitan dengan pembentukan glukosa dari prekursor nonkarbohidrat.
pfa00020	<i>Citrate cycle</i> (TCA atau <i>Krebs cycle</i>)	Jaringan reaksi yang menghubungkan metabolisme karbon dengan pembentukan pembawa elektron dan prekursor biosintesis.
pfa00030	<i>Pentose phosphate pathway</i>	Jalur yang menyediakan gula pentosa dan, pada cabang oksidatif, daya reduksi dalam bentuk NADPH.

Nama *pathway* pada `pathway.csv` dipertahankan persis dari sumber, termasuk akhiran organisme. Nama ringkas dan pengertian pada tabel di atas merupakan pengantar umum, bukan kesimpulan bahwa seluruh reaksi atau arah aliran metabolisme tersebut aktif dalam sampel *P. falciparum* tertentu.

Arti Anotasi Gen ke Pathway

Dalam dataset praktikum, hubungan gen dengan *pathway* disimpan sebagai anotasi. Satu gen dapat terhubung dengan lebih dari satu *pathway*, dan satu *pathway* dapat memiliki anotasi untuk banyak gen. Hubungan yang tercatat tidak dengan sendirinya membuktikan ekspresi gen, aktivitas *pathway*, atau interaksi langsung antargen maupun protein.

2.4 Mengenali Tiga Berkas Data Praktikum

Dataset praktikum disediakan dalam tiga berkas CSV sebagaimana dirangkum pada Tabel 2.

Tabel 2. Isi tiga berkas CSV praktikum.

Berkas	Satu baris mewakili	Jumlah
gene.csv	Satu gen dan rentang genomiknya	39
pathway.csv	Satu <i>pathway</i> dan namanya	3
gene_pathway.csv	Satu pasangan gen- <i>pathway</i>	49

Jumlah pasangan lebih besar daripada jumlah gen karena satu gen dapat berhubungan dengan lebih dari satu *pathway*. Berkas CSV hanya menyimpan nilai dan belum menegaskan *primary key*, *foreign key*, NOT NULL, atau CHECK.

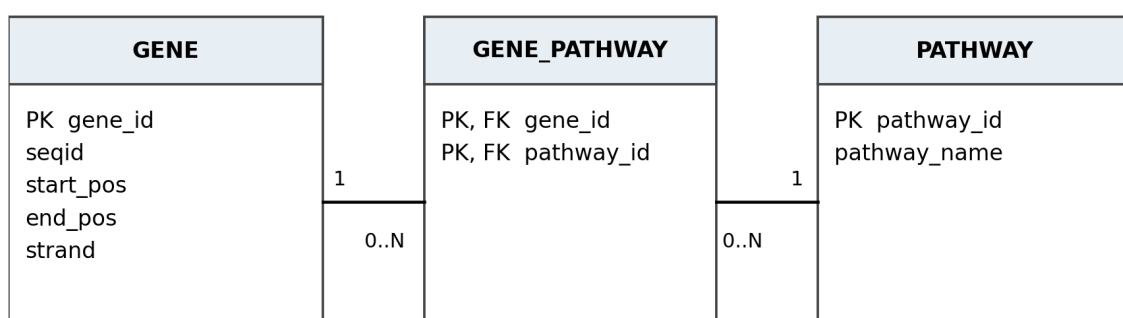
Pada penyiapan data, kolom sumber `genome_version` dipetakan menjadi `seqid`, `start` menjadi `start_pos`, dan `end` menjadi `end_pos`. Hanya fitur `protein_coding_gene` yang dipilih, sehingga setiap gen dalam `gene.csv` direpresentasikan satu kali dan `gene_id` dapat digunakan sebagai *primary key*.

3 Model Data Praktikum

Bagian ini meninjau kembali pemetaan ERD ke model relasional menggunakan dataset biologis yang akan digunakan pada praktikum.

3.1 ERD Praktikum

ERD pada Gambar 1 memuat entitas `gene`, `pathway`, dan entitas penghubung `gene_pathway`. Entitas penghubung menyimpan pasangan gen-*pathway* untuk merepresentasikan hubungan banyak ke banyak.



Gambar 1. ERD katalog gen dan anotasi *pathway* untuk praktikum.

Satu gen dapat mempunyai nol atau banyak anotasi pada tabel `gene_pathway`, dan satu *pathway* dapat mempunyai nol atau banyak anotasi pada tabel yang sama. Setiap baris `gene_pathway` merujuk tepat satu gen dan satu *pathway*. ERD tetap mengizinkan gen belum mempunyai pasangan, meskipun seluruh gen pada kondisi awal data mempunyai sedikitnya satu pasangan karena cara subset dipilih.

Mengapa Memerlukan Tabel Penghubung?

Hubungan *gen-pathway* yang bersifat M:N dipetakan ke tabel **gene_pathway**. Setiap baris menyimpan satu pasangan *gen-pathway* sehingga informasi pada tabel **gene** dan **pathway** tidak perlu diulang. Karena memuat *foreign key*, **gene_pathway** dibuat dan diisi setelah kedua tabel induknya. Tanpa aturan penghapusan otomatis, baris pada tabel induk yang masih dirujuk tidak dapat dihapus sebelum baris rujukannya pada **gene_pathway** dihapus.

3.2 Kamus Data Praktikum

Makna dan aturan seluruh kolom pada ketiga tabel dirangkum pada Tabel 3.

Tabel 3. Kamus data model praktikum.

Kolom	Makna dan aturan
gene.gene_id	Identifier gen berupa TEXT; menjadi PK, unik, dan wajib diisi.
gene.seqid	Label sekuens referensi berupa TEXT; wajib diisi.
gene.start_pos	Batas awal berupa INTEGER; wajib diisi dan lebih besar dari 0.
gene.end_pos	Batas akhir berupa INTEGER; wajib diisi dan tidak lebih kecil dari start_pos.
gene.strand	Arah <i>strand</i> berupa TEXT; wajib diisi dan hanya boleh + atau -.
pathway.pathway_id	Identifier <i>pathway</i> berupa TEXT; menjadi PK, unik, dan wajib diisi.
pathway.pathway_name	Nama <i>pathway</i> berupa TEXT; wajib diisi dan tidak menjadi PK.
gene_pathway.gene_id	Bagian PK gabungan dan FK ke gene (gene_id); wajib diisi.
gene_pathway.pathway_id	Bagian PK gabungan dan FK ke pathway (pathway_id); wajib diisi.

PK gabungan **gene_id** dan **pathway_id** mencegah duplikasi pasangan *gen-pathway*.

4 Tugas Prapraktikum

Setelah mempelajari bagian Pengenalan Data Biologis dan Model Data Praktikum, periksa berkas **gene.csv**, **pathway.csv**, dan **gene_pathway.csv**. Jawab pertanyaan berikut secara individual sebelum mengikuti kegiatan praktikum.

Tugas

1. **Makna satu baris data.** Jelaskan apa yang direpresentasikan oleh satu baris pada masing-masing berkas **gene.csv**, **pathway.csv**, dan **gene_pathway.csv**.
2. **Gen, CDS, dan protein.** Jelaskan perbedaan gen, CDS, dan protein. Mengapa panjang rentang gen yang dihitung dari **start_pos** dan **end_pos** tidak otomatis sama dengan panjang CDS atau protein?
3. **Makna strand.** Jelaskan arti nilai + dan - pada kolom **strand**. Informasi apa yang tidak dapat disimpulkan hanya dari nilai *strand*?
4. **Primary key gabungan.** Jelaskan alasan tabel **gene_pathway** menggunakan gabungan **gene_id** dan **pathway_id** sebagai *primary key*. Data duplikat seperti apa yang dicegah oleh kunci tersebut?

5. **Prediksi penghapusan data.** Prediksi apa yang terjadi jika satu gen dihapus dari tabel **gene** ketika identifier gen tersebut masih dirujuk oleh **gene_pathway**. Jelaskan aturan yang berkaitan dengan prediksi tersebut.

Jawaban ditulis tangan secara singkat menggunakan kalimat sendiri pada selembar kertas utuh dan dibawa untuk diperiksa pada awal praktikum.

5 Konsep SQL Dasar

5.1 Perintah SQL yang Digunakan

Structured Query Language (SQL) digunakan untuk berinteraksi dengan basis data relasional. Pada praktikum ini, perintah SQL dijalankan pada PostgreSQL untuk mengatur struktur dan isi tabel. Fungsi setiap perintah dirangkum pada Tabel 4.

Tabel 4. Perintah SQL dasar yang digunakan dalam praktikum.

Perintah	Objek	Fungsi
CREATE TABLE	Struktur	Membuat tabel baru.
ALTER TABLE	Struktur	Mengubah struktur tabel.
INSERT	Data	Menambahkan baris baru.
SELECT	Data	Membaca atau menampilkan data.
UPDATE	Data	Memperbarui nilai pada baris.
DELETE	Data	Menghapus baris.

5.2 Struktur dan Constraint

Struktur tabel meliputi nama tabel, kolom, tipe data, dan *constraint*, sedangkan isi tabel berupa baris data. **PRIMARY KEY** menjaga identitas tetap unik, **NOT NULL** mewajibkan nilai tersedia, **CHECK** membatasi nilai yang diperbolehkan, dan **FOREIGN KEY** menjaga hubungan antartabel. Perintah SQL akan ditolak apabila data melanggar aturan tersebut.

6 Persiapan Lingkungan Praktikum

6.1 Perangkat dan Ruang Kerja

Praktikum dilaksanakan menggunakan PostgreSQL lokal melalui aplikasi pgAdmin yang telah dipasang dan digunakan pada pertemuan sebelumnya. Perintah SQL ditulis dan dijalankan melalui *Query Tool*. Pastikan layanan PostgreSQL berjalan dan koneksi server yang digunakan pada Pertemuan 3 masih dapat dibuka.

Catatan: Ruang Kerja PostgreSQL

SQL adalah bahasa untuk menulis perintah basis data. PostgreSQL merupakan DBMS atau server yang menyimpan dan mengelola basis data, sedangkan pgAdmin adalah aplikasi klien yang mengirimkan perintah SQL ke PostgreSQL. Di dalam server, *database* menjadi ruang penyimpanan data dan *schema* menjadi ruang nama untuk mengelompokkan tabel. Praktikum menggunakan database **db.pfalciparum.p4** dan *schema* bawaan **public** agar

tabel Pertemuan 4 terpisah dari latihan sebelumnya.

6.2 Berkas Praktikum Mahasiswa

Mahasiswa menggunakan berkas yang telah disediakan, yaitu `ERD_Mg04.png`, `02_data_lanjutan.sql`, `gene.csv`, `pathway.csv`, dan `gene_pathway.csv`. ERD menjadi acuan model, sedangkan skrip SQL digunakan untuk melengkapi data setelah tiga gen pertama dimasukkan secara manual.

6.3 Membuka Query Tool dan Menyiapkan Database

Lakukan persiapan berikut sebelum membuat tabel.

1. Buka pgAdmin dan hubungkan aplikasi ke server PostgreSQL lokal yang digunakan pada Pertemuan 3.
2. Pilih database bawaan `postgres`, lalu buka *Query Tool* melalui menu **Tools**.
3. Pastikan *auto-commit* aktif. Jalankan perintah berikut untuk melihat versi PostgreSQL, kemudian catat versinya tanpa mencantumkan kata sandi atau informasi rahasia.

```
SELECT version();
```

Jalankan perintah berikut sebagai satu eksekusi tersendiri.

```
CREATE DATABASE db_pfalciparum_p4;
```

Setelah berhasil, segarkan daftar **Databases** pada panel pgAdmin. Pilih `db_pfalciparum_p4`, lalu buka *Query Tool* baru. Buat dan simpan berkas `NIM_Mg04.sql`. Tuliskan seluruh perintah yang disusun selama praktikum pada berkas tersebut sesuai urutan kegiatan.

Periksa ruang kerja dengan menjalankan:

```
SELECT current_database(), current_schema();
```

Hasilnya harus menunjukkan database `db_pfalciparum_p4` dan *schema* `public`. Pada panel pgAdmin, buka **Schemas > public > Tables** dan pastikan tabel `gene`, `pathway`, serta `gene_pathway` belum tersedia.

7 Penerapan SQL Dasar

7.1 Membuat Tabel dengan CREATE TABLE

Perintah `CREATE TABLE` digunakan untuk membuat struktur tabel. Jalankan setiap blok berikut secara berurutan pada database yang belum memiliki tabel `gene`, `pathway`, dan `gene_pathway`.

Langkah 1: Membuat Tabel gene

Tabel `gene` dibuat untuk menyimpan identitas gen, lokasi pada sekuens referensi, dan arah *strand*.

```
CREATE TABLE gene (  
    gene_id TEXT PRIMARY KEY,  
    seqid TEXT NOT NULL,  
    start_pos INTEGER NOT NULL CHECK (start_pos > 0),
```

```

end_pos INTEGER NOT NULL,
strand TEXT NOT NULL CHECK (strand IN ('+', '-')),
CONSTRAINT gene_valid_interval
    CHECK (end_pos >= start_pos)
);

```

`gene_id` menjadi *primary key*. NOT NULL mewajibkan nilai terisi, sedangkan CHECK membatasi koordinat dan nilai *strand*. *Constraint gene_valid_interval* memastikan `end_pos` tidak lebih kecil daripada `start_pos`.

Langkah 2: Membuat Tabel `pathway`

Tabel `pathway` menyimpan identitas dan nama *pathway*.

```

CREATE TABLE pathway (
    pathway_id TEXT _____,
    pathway_name TEXT _____
);

```

Lengkapi bagian kosong berdasarkan Tabel 3. Tentukan constraint yang menjadikan `pathway_id` sebagai pengenal unik dan yang mewajibkan `pathway_name` terisi. Jelaskan fungsi setiap constraint sebelum menjalankan perintah.

Langkah 3: Membuat Tabel `gene_pathway`

Tabel `gene_pathway` dibuat setelah kedua tabel induknya tersedia.

```

CREATE TABLE gene_pathway (
    gene_id TEXT NOT NULL
        REFERENCES _____(_____),
    pathway_id TEXT NOT NULL
        REFERENCES _____(_____),
    PRIMARY KEY (_____, _____)
);

```

Lengkapi nama tabel dan kolom rujukan berdasarkan ERD. Setelah itu, tentukan dua kolom penyusun *primary key* gabungan. Periksa bahwa hasilnya mencegah rujukan yang tidak sah dan pencatatan pasangan *gen-pathway* yang sama lebih dari satu kali.

Menjalankan dan Memeriksa Hasil

Sorot dan jalankan setiap blok `CREATE TABLE` satu kali melalui *Query Tool*. Periksa pesan eksekusi sebelum melanjutkan, kemudian perbarui bagian **Tables** pada panel pgAdmin dan pastikan ketiga tabel telah muncul.

Hasil Tahap `CREATE TABLE`

Schema `public` mempunyai tiga tabel kosong, yaitu `gene`, `pathway`, dan `gene_pathway`. Struktur tabel beserta *primary key*, *foreign key*, NOT NULL, dan CHECK telah terbentuk. Data akan dimasukkan pada tahap berikutnya menggunakan `INSERT`.

7.2 Menambahkan Data Awal dengan INSERT

Perintah `INSERT` digunakan untuk menambahkan baris baru ke dalam tabel. Bentuk dasarnya adalah sebagai berikut.

```
INSERT INTO nama_tabel (kolom_1, kolom_2, ...)  
VALUES (nilai_1, nilai_2, ...);
```

`INSERT INTO` menentukan tabel tujuan. Daftar kolom ditulis secara eksplisit, dan urutan nilai harus sesuai dengan urutan kolom. Nilai teks ditulis di dalam tanda kutip tunggal, sedangkan nilai bilangan ditulis tanpa tanda kutip.

Langkah 4: Membaca Tiga Baris Pertama

Buka berkas `gene.csv`, lalu baca tiga baris pertama setelah baris *header*. Gunakan baris dengan `gene_id` PF3D7_0318800, PF3D7_0514600, dan PF3D7_0610800.

Langkah 5: Menulis INSERT secara Mandiri

Ubah ketiga baris tersebut menjadi perintah `INSERT`. Gunakan daftar kolom eksplisit dan lengkapi kerangka berikut untuk setiap gen.

```
INSERT INTO gene (gene_id, seqid, start_pos, end_pos, strand)  
VALUES ('...', '...', ..., ..., '...');
```

Jalankan setiap pernyataan setelah seluruh nilainya diperiksa.

Langkah 6: Memeriksa Tiga Data Awal

Jalankan perintah berikut setelah ketiga data berhasil dimasukkan.

```
SELECT *  
FROM gene;
```

Bandingkan keluaran dengan tiga baris pertama `gene.csv`. Periksa kembali `gene_id`, penulisan `seqid`, koordinat awal dan akhir, serta tanda *strand*. Pada tahap ini, tabel `gene` berisi tiga baris, sedangkan tabel `pathway` dan `gene_pathway` masih kosong.

Langkah 7: Menjalankan Data Lanjutan

Setelah ketiga data awal benar, buka berkas `02_data_lanjutan.sql` melalui pgAdmin, kemudian jalankan seluruh isi skrip satu kali.

Skrip tersebut menambahkan 36 gen lainnya, kemudian 3 *pathway* dan 49 pasangan *gene-pathway*. Tabel `pathway` dan `gene_pathway` harus masih kosong sebelum skrip dijalankan. Jangan menggabungkan rute ini dengan impor CSV lengkap karena dapat menghasilkan data duplikat dan pelanggaran *primary key*.

Memeriksa Kondisi Awal Data (*Baseline*)

Setelah skrip lanjutan berhasil dijalankan, periksa isi ketiga tabel melalui *Query Tool* atau tampilan data pgAdmin. Cocokkan sekurangnya dua baris dari setiap tabel dengan berkas CSV yang sesuai. Kondisi data yang diharapkan ditunjukkan pada Tabel 5.

Tabel 5. Jumlah baris yang diharapkan pada setiap tahap pengisian.

Tahap	gene	pathway	gene_pathway
Setelah INSERT manual	3	0	0
Setelah skrip lanjutan	39	3	49

7.3 Membaca Data dengan SELECT

Perintah **SELECT** digunakan untuk membaca data tanpa mengubah isi tabel. Pada latihan ini, **WHERE** digunakan untuk memilih baris berdasarkan kondisi, **AND** menggabungkan kondisi yang harus dipenuhi secara bersamaan, dan **ORDER BY** mengurutkan hasil.

Sebagai contoh, perintah berikut menampilkan identitas dan koordinat salah satu gen yang telah dimasukkan secara manual.

```
SELECT gene_id, start_pos, end_pos
FROM gene
WHERE gene_id = 'PF3D7_0318800';
```

SELECT menentukan kolom keluaran, **FROM** menentukan tabel sumber, dan **WHERE** memilih baris yang memenuhi kondisi. Nilai teks ditulis dalam tanda kutip tunggal dan titik koma mengakhiri pernyataan.

Tugas

Untuk Q1–Q5, serta Q6 apabila dikerjakan sebagai pengayaan, tentukan terlebih dahulu tabel sumber, kolom keluaran, kondisi pemilihan, dan prediksi hasil. Setelah itu, tulis dan jalankan SQL, bandingkan hasil aktual dengan prediksi, lalu berikan interpretasi dalam satu atau dua kalimat.

Langkah 8: Q1 – Menelusuri Lokasi Gen

Tim menelusuri gen PF3D7_0815900. Tentukan sekuens referensi, koordinat awal, koordinat akhir, dan *strand* gen tersebut. Tampilkan **gene_id**, **seqid**, **start_pos**, **end_pos**, dan **strand**.

Jelaskan mengapa seluruh informasi tersebut perlu dibaca bersama untuk menafsirkan lokasi gen.

Langkah 9: Q2 – Menelusuri Gen dengan Strand Negatif

Tentukan gen dalam subset praktikum yang berada pada Pf3D7_14_v3 dan mempunyai *strand* negatif. Tampilkan **gene_id**, **start_pos**, **end_pos**, dan **strand**, kemudian urutkan hasil berdasarkan **gene_id**. Kedua kondisi harus dipenuhi secara bersamaan.

Jelaskan mengapa hasil tersebut belum memberikan informasi tentang tingkat ekspresi gen.

Langkah 10: Q3 – Menelusuri Gen dalam Interval Target

Tim meninjau interval posisi 800000 sampai 1000000 pada Pf3D7_10_v3. Tentukan gen yang seluruh rentangnya berada di dalam interval tersebut. Kedua posisi batas ikut disertakan. Tampilkan **gene_id**, **start_pos**, dan **end_pos**, kemudian urutkan berdasarkan **start_pos**

dan `gene_id`.

“Seluruh rentang berada di dalam” berarti `start_pos` tidak kurang dari batas bawah dan `end_pos` tidak melebihi batas atas. Kondisi ini berbeda dari tumpang tindih sebagian.

Langkah 11: Q4 – Menelusuri Gen pada Pathway pfa00010

Gunakan tabel `gene_pathway` untuk menentukan seluruh `gene_id` yang dipetakan ke pfa00010. Urutkan hasil berdasarkan `gene_id`. Nama *pathway* telah tersedia pada Tabel 1, sehingga JOIN tidak diperlukan.

Jelaskan apakah daftar tersebut membuktikan bahwa seluruh gen aktif pada kondisi eksperimen yang sama.

Langkah 12: Q5 – Menelusuri Pathway untuk Satu Gen

Tentukan seluruh `pathway_id` yang dipetakan ke PF3D7_0815900 menggunakan tabel `gene_pathway`. Urutkan hasil berdasarkan `pathway_id`, lalu cocokkan identifier yang diperoleh dengan Tabel 1 atau berkas `pathway.csv`.

Jelaskan implikasi hasil tersebut terhadap penggunaan *primary key* gabungan pada tabel `gene_pathway`.

Pengayaan

Langkah 13: Q6 – Menghitung Rentang Genomik

Hitung rentang genomik PF3D7_0815900 menggunakan $\text{end_pos} - \text{start_pos} + 1$, kemudian beri nama kolom keluaran `genomic_span_bp` menggunakan AS. Nyatakan asumsi bahwa koordinat menggunakan sistem *1-based inclusive*.

Jelaskan mengapa hasil perhitungan tersebut tidak dapat langsung dibagi tiga untuk menyimpulkan panjang protein.

7.4 Mengubah Struktur dan Data dengan ALTER TABLE dan UPDATE

Pada kegiatan M1, tim memerlukan tempat untuk menyimpan catatan pemeriksaan setiap gen. Mahasiswa akan menambahkan satu kolom, memeriksa nilainya sebelum diisi, memperbarui catatan untuk satu gen, lalu memeriksa kembali hasilnya.

Langkah 14: M1 – Menambahkan Catatan Pemeriksaan

Menambahkan kolom dengan ALTER TABLE. Bentuk dasar untuk menambahkan kolom adalah sebagai berikut.

```
ALTER TABLE nama_tabel
ADD COLUMN nama_kolom tipe_data;
```

Terapkan bentuk tersebut untuk menambahkan kolom `review_note` bertipe TEXT pada tabel `gene`. Karena kolom ditambahkan tanpa nilai *default*, baris lama memperoleh NULL. NULL berarti nilai belum tersedia dan diperiksa menggunakan IS NULL, bukan = NULL.

Memeriksa nilai sebelum diperbarui. Tampilkan `gene_id` dan `review_note` untuk gen

PF3D7_0815900. Catat hasilnya sebelum melanjutkan ke UPDATE.

Memperbarui satu baris dengan UPDATE. Bentuk dasar perintahnya adalah:

```
UPDATE nama_tabel
SET nama_kolom = nilai_baru
WHERE kondisi;
```

Susun dan jalankan perintah untuk mengisi `review_note` milik PF3D7_0815900 dengan teks 'Koordinat cocok dengan snapshot BioSQLite'. Setelah itu, tampilkan kembali `gene_id` dan `review_note` gen tersebut untuk memastikan perubahan berhasil.

Catatan: Perhatian saat UPDATE

Pastikan `WHERE` memilih PF3D7_0815900; tanpa `WHERE`, seluruh baris akan diperbarui.

Perubahan nilai yang diharapkan pada setiap tahap dirangkum pada Tabel 6.

Tabel 6. Perubahan kolom `review_note` selama kegiatan M1.

Tahap	PF3D7_0815900	Baris lain
Sebelum ALTER TABLE	Kolom belum tersedia	Kolom belum tersedia
Sesudah ALTER TABLE	NULL	NULL
Sesudah UPDATE	Catatan pemeriksaan terisi	Tetap NULL

Tugas

Jelaskan perubahan struktur yang dilakukan oleh `ALTER TABLE`, alasan baris lama memperoleh `NULL`, perubahan data yang dilakukan oleh `UPDATE`, dan fungsi kondisi `WHERE`. Bedakan perubahan struktur tabel dari perubahan isi tabel.

7.5 Menambahkan Data Simulasi dan Hubungannya

Kegiatan M2 menggunakan kembali `INSERT` untuk menambahkan data pada dua tabel yang saling berhubungan. Fokus kegiatan ini adalah urutan pemasukan data induk dan data anak yang diatur oleh *foreign key*.

Langkah 15: M2 – Menambahkan Gen Simulasi

Tambahkan satu baris ke tabel `gene` dengan `gene_id` `SIMULASI_001`, `seqid` `SIMULASI_SEQ`, `start_pos` 100, `end_pos` 200, dan `strand` `+`. Susun `INSERT` dengan daftar kolom eksplisit. Kolom `review_note` boleh tidak dicantumkan dan akan bernilai `NULL`.

Setelah gen berhasil dimasukkan, tambahkan pasangan pada tabel `gene_pathway` menggunakan `gene_id` `SIMULASI_001` dan `pathway_id` `pfa00010`. `SIMULASI_001` merupakan data buatan untuk mempelajari operasi SQL dan integritas referensial, bukan gen atau anotasi biologis nyata.

Catatan: Urutan Pemasukan Data

Masukkan `SIMULASI_001` ke tabel `gene` terlebih dahulu, kemudian masukkan pasangannya ke `gene_pathway`. Urutan sebaliknya ditolak oleh *foreign key* karena gen tersebut belum tersedia pada tabel induk. `pfa00010` sudah tersedia pada tabel `pathway`.

Memeriksa Hasil

Gunakan `SELECT` dengan kondisi `gene_id = 'SIMULASI_001'` pada tabel `gene` dan `gene_pathway`. Pastikan tabel `gene` memuat gen simulasi, `gene_pathway` memuat pasangannya dengan `pfa00010`, dan tabel `pathway` tidak berubah.

Tugas

Jelaskan mengapa gen harus dimasukkan sebelum pasangan anotasinya dan *constraint* yang mengatur urutan tersebut.

7.6 Menghapus Data dengan DELETE

Perintah `DELETE` digunakan untuk menghapus baris tanpa menghapus struktur tabel. Bentuk dasarnya adalah sebagai berikut.

```
DELETE FROM nama_tabel  
WHERE kondisi;
```

Kegiatan M3 menggunakan data simulasi dari M2 untuk memperlihatkan akibat *foreign key* terhadap urutan penghapusan.

Langkah 16: M3 – Menghapus Data yang Masih Dirujuk

Sebelum menjalankan perintah, prediksi apa yang terjadi jika `SIMULASI_001` langsung dihapus dari tabel `gene` ketika pasangan anotasinya masih tersedia pada tabel `gene_pathway`. Catat prediksi dan alasannya.

Susun perintah `DELETE` yang mencoba menghapus gen tersebut dari tabel `gene`, lalu jalankan sebagai satu pernyataan tersendiri. Amati pesan PostgreSQL dan bandingkan hasilnya dengan prediksi.

Baris induk seharusnya tidak terhapus karena masih dirujuk oleh *foreign key* pada tabel `gene_pathway`. Pesan kesalahan perlu dibaca untuk mengenali tabel yang masih menyimpan rujukan dan *constraint* yang menyebabkan penolakan.

Menghapus dengan Urutan yang Benar

Hapus pasangan `SIMULASI_001` dari tabel `gene_pathway`, kemudian hapus gen tersebut dari tabel `gene`. Gunakan kondisi `WHERE` yang hanya memilih `SIMULASI_001` dan jangan menggunakan `CASCADE`.

Catatan: Perhatian saat DELETE

Pastikan `WHERE` hanya memilih `SIMULASI_001`; tanpa `WHERE`, seluruh baris pada tabel akan dihapus.

Memeriksa Hasil Akhir

Gunakan **SELECT** untuk mencari **SIMULASI_001** pada tabel **gene** dan **gene_pathway**. Kedua query seharusnya tidak menghasilkan baris. Pastikan data asli kembali memuat 39 gen, 3 *pathway*, dan 49 pasangan.

Tugas

Jelaskan alasan penghapusan pertama ditolak, alasan tabel anak harus dihapus lebih dahulu, dan risiko **DELETE** tanpa **WHERE**.

7.7 Menguji Integrity Constraint

Pada kegiatan M4, mahasiswa sengaja mencoba memasukkan data yang tidak memenuhi aturan tabel. Pesan kesalahan merupakan hasil yang diharapkan karena menunjukkan bahwa *constraint* bekerja.

Langkah 17: M4 – Memeriksa Aturan Data

Untuk setiap percobaan, tuliskan prediksi sebelum menjalankan SQL. Jalankan satu pernyataan pada satu waktu dengan *auto-commit* aktif dan di luar blok **BEGIN**. Amati pesan PostgreSQL, tentukan *constraint* yang menyebabkan penolakan, dan pastikan *baseline* tidak berubah. Gunakan identifier baru dan buat hanya nilai yang sedang diuji menjadi tidak valid agar PostgreSQL tidak lebih dahulu menemukan pelanggaran lain.

1. **Primary key tabel gene.** Coba masukkan gen menggunakan **gene_id** PF3D7_0815900 yang sudah tersedia. Buat seluruh nilai lainnya valid agar pengujian menyasar PK duplikat.
2. **NOT NULL.** Gunakan **gene_id** baru **UJI_NULL**, tetapi berikan **NULL** pada **seqid**. Buat kolom lainnya valid.
3. **CHECK interval koordinat.** Gunakan **gene_id** **UJI_INTERVAL** dengan **start_pos** bernilai 20 dan **end_pos** bernilai 10.
4. **CHECK strand.** Gunakan **gene_id** **UJI_STRAND** dengan nilai ? pada **strand**. Buat koordinat dan kolom lainnya valid.
5. **Foreign key.** Coba masukkan pasangan **GEN_TIDAK_ADA** dan **pfa00010** ke tabel **gene_pathway**. **pfa00010** tersedia, tetapi identifier gen tersebut tidak tersedia pada tabel **gene**.
6. **Primary key gabungan.** Coba masukkan kembali pasangan **PF3D7_0815900** dan **pfa00010** yang sudah tersedia pada tabel **gene_pathway**.

Tugas

Untuk setiap percobaan, tuliskan prediksi sebelum menjalankan SQL. Setelah perintah dijalankan, catat hasil aktual PostgreSQL serta *constraint* yang menyebabkan data ditolak pada Tabel 7.

Tabel 7. Prediksi dan hasil pengujian *constraint*.

Percobaan	Prediksi	Hasil aktual	Constraint dan penjelasan
PK tabel <i>gene</i>			
NOT NULL			
CHECK interval			
CHECK strand			
<i>Foreign key</i>			
PK gabungan			

Catatan: Penyimpanan Percobaan Gagal

Simpan perintah yang sengaja gagal sebagai komentar dalam `NIM_Mg04.sql` agar skrip utama dapat dijalankan kembali dari awal.

Memeriksa Kondisi Akhir

Setelah seluruh percobaan selesai, pastikan tidak ada data uji yang tersimpan. Tabel *gene*, *pathway*, dan *gene_pathway* masing-masing berisi 39, 3, dan 49 baris. Tidak ada `UJI_NULL`, `UJI_INTERVAL`, `UJI_STRAND`, `GEN_TIDAK_ADA`, atau `SIMULASI_001`. Kolom *review_note* tetap tersedia dan hanya catatan milik `PF3D7_0815900` yang terisi.

8 Latihan Individual dan Refleksi

Bagian ini memeriksa kemampuan mahasiswa menerapkan kembali pola query yang telah dipelajari. Tidak ada tabel atau data baru yang ditambahkan; latihan menggunakan data pada tabel *gene* yang telah terisi pada Bagian 7. Query tantangan dicantumkan dalam `NIM_Mg04.sql`, sedangkan prediksi, hasil, interpretasi, dan refleksinya dicantumkan dalam laporan.

8.1 Tantangan Individual

Q2 sebelumnya mencari gen dengan *strand* negatif pada `Pf3D7_14.v3`. Tantangan berikut menggunakan pola yang sama, tetapi *seqid* diubah menjadi `Pf3D7_10.v3`.

Tugas

Tentukan gen pada `Pf3D7_10.v3` yang mempunyai *strand* negatif. Tampilkan *gene_id*, *start_pos*, *end_pos*, dan *strand*, kemudian urutkan hasil berdasarkan *gene_id*.

1. Tuliskan tabel sumber, kolom keluaran, dan dua kondisi yang diperlukan.
2. Catat prediksi jumlah baris yang akan diperoleh.
3. Susun dan jalankan SQL secara mandiri.
4. Catat jumlah baris dan *gene_id* yang muncul.

5. Berikan interpretasi singkat terhadap hasil tersebut.

8.2 Refleksi Prediksi dan Hasil

Gunakan tantangan pada Bagian 8.1 untuk membandingkan prediksi dengan hasil aktual. Catat informasi tersebut pada Tabel 8. Jika hasilnya berbeda, jelaskan letak kekeliruan atau asumsi yang diperbaiki. Jika hasilnya sama, jelaskan alasan prediksi tersebut sesuai dengan hasil query.

Tabel 8. Refleksi prediksi dan hasil pelaksanaan SQL.

Komponen	Catatan
Kegiatan atau perintah	
Prediksi	
Hasil aktual	
Penjelasan perbedaan dan pemahaman yang diperoleh	

9 Lembar Kerja Praktikum

9.1 Berkas yang Dikumpulkan

Setiap mahasiswa mengumpulkan berkas pada Tabel 9.

Tabel 9. Berkas yang dikumpulkan.

Berkas	Isi utama
NIM_Mg04.sql	Perintah SQL yang disusun selama praktikum.
NIM_Laporan_Mg04.pdf	Dokumentasi hasil dan pembahasan praktikum.

9.2 Isi Skrip SQL

Susun NIM_Mg04.sql sesuai urutan kegiatan dan beri komentar untuk menandai setiap tahap. Skrip memuat pembuatan ketiga tabel, pemasukan data, Q1–Q5, Q6 apabila dikerjakan sebagai pengayaan, kegiatan M1–M4, serta query tantangan individual. Beri komentar pada posisi ketika 02_data_lanjutan.sql dijalankan. Perintah yang sengaja menghasilkan kesalahan ditulis sebagai komentar dalam NIM_Mg04.sql agar tidak menghentikan skrip utama.

9.3 Struktur Laporan Praktikum

Laporan disusun sebagai dokumentasi penerapan SQL dasar pada data gen dan anotasi *pathway*. Struktur dan isi minimal laporan dirangkum pada Tabel 10.

Tabel 10. Struktur laporan praktikum SQL dasar

No.	Komponen	Isi Minimal
1	Cover	Judul laporan, identitas mata kuliah, nama dan NIM mahasiswa, program studi, fakultas, universitas, serta tahun akademik.
2	Daftar Isi	Seluruh judul dan subjudul laporan beserta nomor halaman.
3	Model Data	ERD serta penjelasan <i>primary key</i> , <i>foreign key</i> , dan <i>primary key</i> gabungan.
4	Penerapan SQL Dasar	SQL, bukti hasil yang relevan, dan interpretasi Q1–Q5 serta kegiatan M1–M3. Q6 dicantumkan apabila dikerjakan sebagai pengayaan.
5	Pengujian Integritas	Dokumentasi M4 yang memuat data uji, prediksi, hasil aktual PostgreSQL, aturan yang diuji, serta penjelasan pesan penolakan.
6	Tantangan Individual dan Refleksi	Kondisi yang diberikan, prediksi, SQL, hasil aktual, interpretasi, serta refleksi pada Tabel 8.
7	Pembahasan	Jelaskan alasan jumlah pasangan dapat melebihi jumlah gen; gen pada <i>pathway</i> yang sama belum tentu berinteraksi langsung; <i>strand</i> bukan ukuran ekspresi; dan peran FK dalam mencegah rujukan tidak sah.
8	Daftar Pustaka	Sumber yang benar-benar dirujuk, bila digunakan.

Untuk setiap pertanyaan, sertakan SQL dan interpretasi dalam satu atau dua kalimat. Tangkapan layar hanya menjadi bukti pendukung.

9.4 Ketentuan Teknis

Penulisan laporan mengikuti PPKI. Ketentuan teknisnya dirangkum pada Tabel 11.

Tabel 11. Ketentuan teknis laporan praktikum SQL dasar

Komponen	Ketentuan
Format berkas	Laporan dikumpulkan dalam format PDF dan skrip dalam format <code>.sql</code> .
Margin halaman	Kiri 4 cm; atas, kanan, dan bawah masing-masing 3 cm.
Teks isi	Ukuran huruf 12 pt dengan spasi 1,5.
Judul bagian	Ukuran huruf judul bagian 14 pt.
Daftar isi	Memuat judul, subjudul, dan nomor halaman.
Gambar dan tangkapan layar	Memiliki nomor dan <i>caption</i> di bawah gambar serta terbaca pada ukuran halaman.
Tabel	Memiliki nomor dan <i>caption</i> di atas tabel; tabel lintas halaman memakai kepala tabel berulang dan judul “(lanjutan)”.
Skrip dan keamanan	Percobaan yang sengaja gagal ditulis sebagai komentar dalam skrip utama. Kata sandi dan kredensial koneksi tidak dicantumkan. Nama database, <i>schema</i> , dan versi PostgreSQL boleh dicatat tanpa informasi rahasia.

10 Daftar Pustaka

- [1] NCBI Datasets. *GFF3 format: Column Specifications*. Dokumentasi konsep koordinat, *strand*, dan tingkat fitur. <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/file-formats/annotation-files/about-ncbi-gff3/>. Diakses 10 September 2026.
- [2] Kyoto Encyclopedia of Genes and Genomes. *KEGG PATHWAY Database*. <https://www.genome.jp/kegg/pathway.html>. Diakses 10 September 2026.
- [3] PostgreSQL Global Development Group. *PostgreSQL Documentation: Constraints*. <https://www.postgresql.org/docs/current/ddl-constraints.html>. Diakses 10 September 2026.
- [4] PostgreSQL Global Development Group. *PostgreSQL Documentation: Transactions*. Dokumentasi Transactions PostgreSQL. Diakses 10 September 2026.
- [5] BioSQLite. Dokumentasi dan snapshot basis data biologis yang menyertai paket praktikum. <https://github.com/roelof89/biosqlite>.